



SHOPSPHERE

E-COMMERCE DATABASE

OCT 2026

Complete Project Report

Relational database design • MySQL 8+ • ER modelling • Normalization • SQL analytics



Contribution

Sl. No.	Team Member	Contribution / Responsibility
1	Afifa	Worked on writing and implementing SQL transition queries.
2	Bhuvan	Prepared the complete README and handled the overall project report preparation and documentation.
3	Darsh	Created the database and schema, pushed the project files to GitHub, and worked on testing, debugging, and resolving failures.
4	Fayaz	Designed and prepared the ER (Entity-Relationship) diagrams for the database.
5	Gokul	Worked on database normalization in collaboration with Hemanth.
6	Hemanth	Worked on normalization and performed testing and verification of all SQL queries.



Project Components

Project Component	Details
Database	ShopSphere
Implementation	MySQL 8+
Tables	11
Sample Records	150
Business Queries	10
Validation Checks	14



Table of Contents

Section	Topic
1	Project Overview
2	Objectives and Technology
3	Database Architecture
4	Entity and Table Documentation
5	Relationships and Keys
6	Normalization and Integrity
7	Dataset Summary
8	Business Questions and Analytics
9	Validation and Quality Checks
10	Project Strengths
11	Future Enhancements
12	Conclusion

Implementation note

The supplied project image describes the schema as a PostgreSQL project, while the supplied SQL uses MySQL-specific constructs such as **USE shoppsphere** and **AUTO_INCREMENT**, and the business-query section explicitly states MySQL 8+. This report documents the actual supplied SQL implementation as MySQL 8+.



1. Project Overview

ShopSphere is a relational database for an e-commerce platform. It models the major stages of the order lifecycle: customer management, addresses, product catalogue, categories, inventory, orders, order items, payments, shipments, reviews and suppliers.

The supplied dataset contains 150 sample rows distributed across 11 tables. The project also includes analytical SQL for business questions and a dedicated validation script for duplicate and referential-integrity checks.

Core capabilities

- Store customer and address information.
- Organize products using categories and parent categories.
- Track product prices and inventory.
- Record customer orders and individual order items.
- Track payments and their status.
- Track shipments and delivery dates.
- Store and analyze product reviews.
- Represent product–supplier many-to-many relationships.
- Answer practical business questions with SQL.
- Validate normalization and referential integrity.



2. Objectives and Technology

Objectives

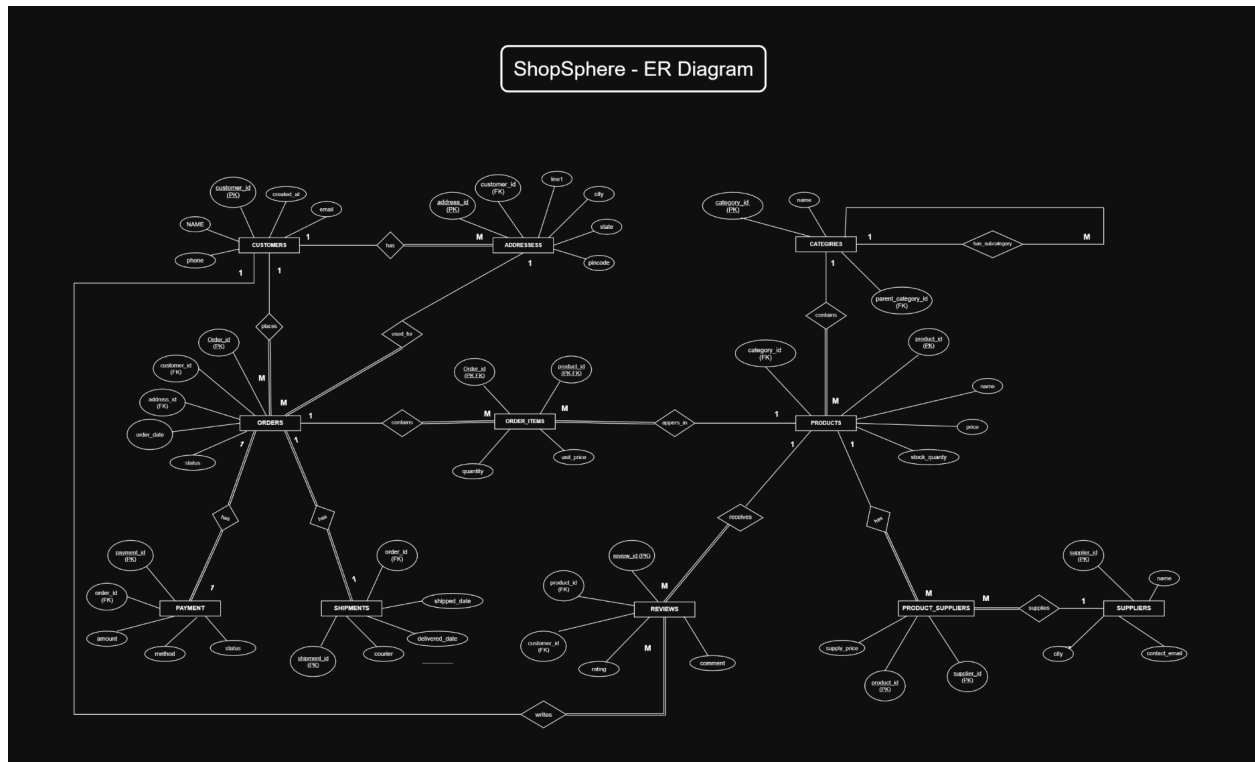
- Design a structured relational database for an e-commerce platform.
- Minimize redundancy through normalized entity separation.
- Enforce data consistency with primary keys, foreign keys and constraints.
- Provide a realistic sample dataset for analysis.
- Demonstrate practical SQL joins and aggregate queries.
- Validate the resulting database for common integrity problems.

Technology and concepts

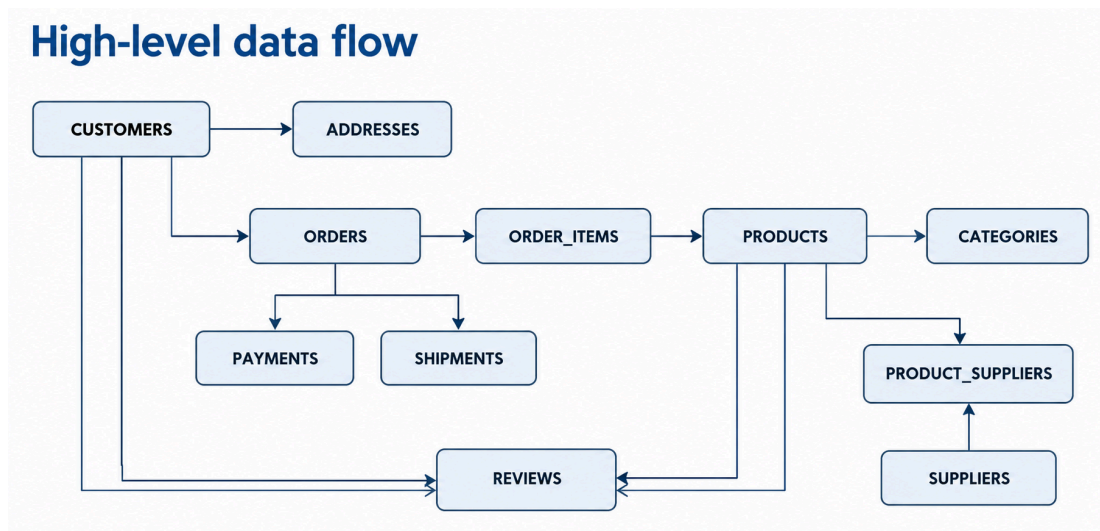
Component	Implementation
Database	MySQL 8+
Model	Relational
Query Language	SQL
Schema size	11 tables
Sample data	150 rows
Normalization	3NF-oriented relational design
Constraints	PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK
Analytics	JOIN, GROUP BY, HAVING, ORDER BY, SUM, AVG, COUNT, ROUND

3. Database Architecture

The architecture separates the core business entities into independent tables and connects them using foreign keys. Many-to-many relationships are resolved through junction tables.



High-level data flow





4. Entity and Table Documentation

Table	Main Attributes	Purpose	Rows
CUSTOMERS	customer_id, name, email, phone, created_at	Customer accounts and contact information.	12
ADDRESSES	address_id, customer_id, line1, city, state, pincode	Addresses belonging to customers.	13
CATEGORIES	category_id, name, parent_category_id	Product categories and hierarchy.	7
PRODUCTS	product_id, name, category_id, price, stock_qty	Catalogue products, prices and inventory.	16
SUPPLIERS	supplier_id, name, contact_email, city	Supplier information.	6
PRODUCT_SUPPLIERS	product_id, supplier_id, supply_price	Product-supplier junction table.	10
ORDERS	order_id, customer_id, address_id, order_date, status	Customer orders and current status.	16
ORDER_ITEMS	order_id, product_id, quantity, unit_price	Products contained in each order.	23
PAYMENTS	payment_id, order_id, amount, method, status	Payment information.	16
SHIPMENTS	shipment_id, order_id, courier, shipped_date, delivered_date	Shipment and delivery information.	16
REVIEWS	review_id, customer_id, product_id, rating, comment	Customer ratings and comments.	11

Important constraints

- Customers: customer_id is the primary key and email is unique.
- Products: price and stock_qty cannot be negative.
- Order items: quantity must be positive and unit_price cannot be negative.
- Payments: amount cannot be negative and status is restricted to defined values.
- Reviews: rating must be between 1 and 5; a customer can review a product only once.
- Junction tables use composite primary keys to prevent duplicate relationships.



5. Relationships and Keys

Relationship summary

Relationship	Cardinality	Purpose
Customers → Addresses	1 : M	A customer may have multiple addresses.
Customers → Orders	1 : M	A customer may place multiple orders.
Addresses → Orders	1 : M	An address may be used by multiple orders.
Categories → Products	1 : M	A category can contain multiple products.
Categories → Categories	1 : M	Parent categories can contain child categories.
Orders → Order Items	1 : M	An order contains multiple line items.
Products → Order Items	1 : M	A product can occur in many order lines.
Orders → Payments	1 : 1	One payment record per order in this design.
Orders → Shipments	1 : 1	One shipment record per order in this design.
Customers → Reviews	1 : M	A customer may write multiple reviews.
Products → Reviews	1 : M	A product may receive multiple reviews.
Products ↔ Suppliers	M : N	Resolved using product_suppliers.

Primary keys

Table	Primary Key
Customers	customer_id
Addresses	address_id
Categories	category_id
Products	product_id
Suppliers	supplier_id
Orders	order_id
Order Items	(order_id, product_id)
Payments	payment_id
Shipments	shipment_id
Reviews	review_id
Product Suppliers	(product_id, supplier_id)



6. Normalization and Integrity

First Normal Form (1NF)

The design separates repeating and multi-valued information into related records. Order lines, addresses, reviews and supplier relationships are stored in their own tables rather than as repeated columns.

Second Normal Form (2NF)

Junction tables use composite keys where the non-key attributes depend on the complete relationship. For example, `order_items` uses (`order_id`, `product_id`) and `product_suppliers` uses (`product_id`, `supplier_id`).

Third Normal Form (3NF)

Independent entities are separated so that customer, address, product, category and supplier attributes are not repeatedly stored inside order or line-item records. Orders reference customers and addresses using IDs, while `order_items` references products using product IDs.

Integrity mechanisms

Mechanism	Examples in ShopSphere
Entity integrity	Primary keys on all entity tables.
Referential integrity	Foreign keys between customers, addresses, orders, products, reviews, suppliers and related tables.
Domain integrity	CHECK constraints on prices, quantities, stock and ratings.
Uniqueness	Unique customer emails; unique customer/product reviews.
Cascading rules	CASCADE, RESTRICT and SET NULL used according to relationship needs.

Validation philosophy

The supplied validation script checks duplicate order items, invalid customer references, address/customer mismatches, orphan records and duplicate payments, shipments, reviews and product-supplier relationships. The expected clean state is zero rows returned by each validation query.



7. Dataset Summary

The supplied insertion script defines 150 sample rows across all 11 tables.

Table	Rows
Customers	12
Addresses	13
Categories	7
Products	16
Suppliers	6
Product Suppliers	10
Orders	16
Order Items	23
Payments	16
Shipments	16
Reviews	11
TOTAL	150

Sample domain coverage

- Categories include Electronics, Fashion, Home & Kitchen and Books, with Mobiles, Laptops and Audio as child categories under Electronics.
- Products include phones, laptops, audio devices, fashion products, home appliances and books.
- Order statuses include Delivered, Shipped, Confirmed, Pending and Cancelled.
- Payment methods include UPI, Card and Net Banking.
- Shipment records include courier, shipped date and optional delivered date.
- Reviews use a 1–5 rating scale and contain customer comments.



8. Business Questions and Analytics

The project contains ten business questions. The supplied assumptions state that cancelled orders are excluded from sales/revenue calculations, only Paid payments count toward customer value, low inventory means `stock_qty < 10`, and a product needs at least two reviews to appear in the highest-rated-products query.

Q1 — Best-selling products

Uses `SUM(quantity)` across `order_items`, joins orders to exclude cancelled orders, then sorts by total units sold.

Product	Units Sold
Wireless Headphones	4
Mens T-Shirt	4
Bluetooth Speaker	3
Running Shoes	3
Python Programming Book	3
Samsung Galaxy S24	2
Data Science Handbook	2

Q2 — Most valuable customers

Ranks customers by total amount paid on non-cancelled orders, considering only payment records with `status = Paid`.

Customer	Total Paid
Aarav Sharma	■140,494
Riya Nair	■74,096
Arjun Kumar	■67,998
Rahul Verma	■58,999
Karan Mehta	■55,696

Q3 — Revenue by category

Revenue is calculated as quantity × unit_price for non-cancelled orders.

Category	Revenue
Mobiles	■224,996
Laptops	■176,997
Audio	■17,993
Fashion	■12,192
Home & Kitchen	■9,498
Books	■5,095

Q4 — Average order value

The query first calculates each non-cancelled order total and then averages those order totals. **Average order value:** ■34,367.00

Q5 — Low inventory

The low-inventory threshold is stock_qty < 10. Products meeting the condition include HP Pavilion Laptop, Coffee Maker, Dell Inspiron Laptop, Smart Watch, Lenovo IdeaPad Slim, iPhone 15 and Air Fryer.

Q6 — Highest-rated products

Product	Average Rating	Reviews
Samsung Galaxy S24	4.67	3
Wireless Headphones	4.67	3
Running Shoes	3.50	2

Q7 — Customers with no orders

A LEFT JOIN identifies customers whose order_id is NULL. The sample identifies Dev Malhotra and Nisha Menon.

Q8 — Orders by status

Status	Order Count
Delivered	8
Shipped	3
Confirmed	2
Pending	2
Cancelled	1

Q9 — Orders awaiting payment

Order	Customer	Amount	Method
5	Ananya Rao	■5,097	Card
16	Vikram Singh	■1,499	Card

Q10 — Supplier/product relationships

The query joins suppliers, product_suppliers and products to report supplier, product and supply price. This demonstrates practical use of a many-to-many relationship.



9. Validation and Quality Checks

The dedicated normalization-validation script contains 14 checks.

#	Validation Check
1	Duplicate order items
2	Invalid customer references
3	Address/customer consistency
4	Orphan products in order items
5	Orphan orders in order items
6	Orphan categories
7	Orphan shipments
8	Orphan payments
9	Orphan reviews
10	Orphan product-supplier relationships
11	Duplicate shipments
12	Duplicate payments
13	Duplicate reviews
14	Duplicate product-supplier relationships

Expected clean-state result

Each validation query is designed to return zero rows when the corresponding integrity condition is satisfied. This provides a simple, repeatable database-quality check after data loading or modification.

Row-count validation

The insertion script includes queries that count rows in every table, calculate the total number of rows, compare the total against a target of 150, and return a PASS message when the total is at least 150.



10. Project Strengths

- Clear separation of business entities into dedicated relational tables.
- Primary keys and foreign keys are defined consistently.
- Composite keys are used appropriately in junction tables.
- Many-to-many product/supplier relationships are normalized through product_suppliers.
- Category hierarchy is represented using a self-referencing foreign key.
- CHECK constraints protect prices, stock, quantities, payments and ratings.
- Unique constraints prevent duplicate emails and duplicate customer/product reviews.
- Analytical queries cover products, customers, revenue, inventory, ratings, orders, payments and suppliers.
- Validation queries explicitly check for orphan records and duplicate relationships.
- The dataset is large enough to demonstrate joins and aggregation while remaining easy to inspect.

Key SQL concepts demonstrated

Concept	Examples
DDL	CREATE DATABASE, CREATE TABLE, DROP DATABASE
DML	INSERT INTO
DQL	SELECT
Joins	INNER JOIN, LEFT JOIN
Aggregation	SUM, AVG, COUNT, ROUND
Grouping	GROUP BY, HAVING
Filtering	WHERE
Sorting	ORDER BY
Constraints	PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK



11. Future Enhancements

Shopping cart. Add CARTS and CART_ITEMS to model items before checkout.

Multiple payment attempts. Remove the one-payment-per-order limitation if the application needs retries, partial payments or multiple transactions.

Discounts and coupons. Add product promotions and order-level coupon relationships.

Returns and refunds. Add return and refund entities to support post-purchase workflows.

Delivery tracking. Add tracking numbers and shipment status history.

Product media. Add a separate product-images table.

Advanced analytics. Add monthly revenue, repeat-customer, margin, delivery-time and inventory-turnover analysis.

Authentication. Add a dedicated account/authentication layer if the database becomes part of a production application.



12. Conclusion

ShopSphere provides a complete relational representation of an e-commerce platform, covering customers, addresses, products, categories, orders, order items, payments, shipments, reviews and suppliers. The schema contains 11 tables and the supplied sample dataset contains 150 rows.

The project demonstrates core DBMS principles: entity modelling, relationship design, primary and foreign keys, composite keys, normalization, referential integrity, domain constraints, aggregation, joins and business-oriented SQL analysis.

The accompanying validation script strengthens the project by checking for duplicate and orphan relationships, while the business-query section shows how the same relational model can support practical operational and analytical questions.

Final project snapshot

Metric	Value
Database	ShopSphere
Implementation	MySQL 8+
Tables	11
Sample rows	150
Business questions	10
Validation checks	14
Primary-key strategy	Single and composite keys
Core relationship types	1:M, 1:1 and M:N
Normalization target	3NF-oriented design